

Silly Little Delivery Guys

A Genetic Algorithm for Multi-Vehicle Delivery Routing

Course Project Report
Fuzzy Systems & Genetic Algorithms

Topic:
Multi-Vehicle Routing Problem (mVRP) optimized with a Genetic
Algorithm
Implementation: Python (NumPy + Pygame)

إعداد الطلاب:

خليل محمد عربي كاتبي

محمد محمد زياد المجذوب

هشام محمد همام الزين

محمود زياد رمزي

محمد غسان محمد خير الخاني

Contents

1. Abstract
2. Introduction
3. Problem Definition
4. Genetic Algorithm Design
 - 4.1 Chromosome Encoding
 - 4.2 Fitness Function
 - 4.3 Selection
 - 4.4 Crossover Operators
 - 4.5 Mutation Operators
 - 4.6 Evolutionary Loop & Elitism
5. Implementation & Architecture
6. Visualization (Race Mode)
7. Configuration Parameters
8. Validation & Results
9. Conclusion
10. References

1. Abstract

This report presents a genetic algorithm (GA) that solves a multi-vehicle delivery routing problem: assigning 100 delivery stops among 5 couriers and ordering each courier's route so that all deliveries finish as early as possible. The optimization objective is the makespan (the length of the slowest courier's route) rather than total distance, which makes the problem a load-balancing as well as a routing task. The solution uses a two-gene chromosome (stop–courier assignment plus a global visit permutation), tournament selection, two crossover operators, three mutation operators including a linkage-aware relocate, and a generational loop with fitness elitism and an optional crossover-credit breeder elitism scheme. The system is implemented in pure NumPy/Python (no machine-learning libraries) and includes a Pygame “race mode” in which the best candidate solutions are animated as live courier races, so that the on-screen finishing order matches solution fitness.

2. Introduction

The Vehicle Routing Problem (VRP), introduced by Dantzig and Ramser (1959), generalizes the Travelling Salesman Problem to multiple vehicles serving a common depot. It is NP-hard: the number of ways to partition stops among couriers and then order each partition grows factorially, so exhaustive search is infeasible even for modest problem sizes. Genetic algorithms are a natural fit because they explore many candidate routings in parallel and recombine partial solutions, while being indifferent to the non-convex, combinatorial structure of the search space.

This project frames the problem as “Silly Little Delivery Guys”: a fleet of couriers leaves a central depot, each serves a subset of customers, and returns. The goal is to minimize how long the whole delivery round takes, which corresponds to the slowest courier. The contribution of the work is twofold: (1) a carefully designed two-gene encoding with operators that respect the coupling between the two genes, and (2) an

intuitive real-time visualization that turns the abstract fitness value into a literal race.

3. Problem Definition

Given a depot located at the center of a 1000×1000 world and a set of N delivery stops, partition the stops among K couriers and determine the visit order of each courier such that the makespan is minimized.

Formally:

- Each courier route starts and ends at the depot.
- Route length = Euclidean distance of depot → ordered stops → depot.
- Makespan = the maximum route length across the K couriers.
- Objective: minimize makespan; total distance is used only as a tie-breaker.

Stops are generated from Gaussian clusters (a `clustered_map` of N stops drawn around several random centers), which mimics realistic delivery demand that concentrates in neighborhoods. A separate `sanity_map` places one tight cluster per courier so the optimal answer is obvious (one courier per cluster), providing a visual correctness check. The default instance is N = 100 stops, K = 5 couriers.

```
def clustered_map(n_stops=100, n_clusters=6, seed=0,
spread=70.0):
    """Stops drawn from Gaussian clusters; depot at center."""
    centers = rng.uniform(0.15 * WORLD, 0.85 * WORLD,
size=(n_clusters, 2))
    which = rng.integers(0, n_clusters, size=n_stops)
    stops = centers[which] + rng.normal(0, spread,
size=(n_stops, 2))
    depot = np.array([WORLD / 2, WORLD / 2])
    return depot, np.clip(stops, 0.02 * WORLD, 0.98 * WORLD)
```

Listing 1 — Clustered map generator (delivery_ga/domain/maps.py).

4. Genetic Algorithm Design

4.1 Chromosome Encoding

Each individual carries two genes that together describe a complete routing plan:

- assignment — an integer array of length N, value-encoded: assignment[i] is the courier that owns stop i.
- order — a permutation of all N stop indices: the global visit sequence.

Decoding is elegant: a courier visits its owned stops in the relative order in which they appear in the global order gene. This guarantees every decoded solution is feasible (every stop is served exactly once) regardless of how the genes are recombined.

```
def decode(assignment, order, n_couriers):  
    """Chromosome -> list of route stop indices per  
    courier."""  
    owner_in_seq = assignment[order]  
    return [order[owner_in_seq == courier] for courier in  
            range(n_couriers)]
```

Listing 2 — Chromosome decode (delivery_ga/domain/routing.py).

The two genes are coupled (epistatic): changing a stop's owner without moving it in the order, or vice-versa, can be disruptive. Section 4.5 introduces a relocate operator designed specifically to handle this coupling.

4.2 Fitness Function

Fitness is the reciprocal of a cost that is dominated by the makespan, with a small weighted total-distance term acting only as a tie-breaker between equally balanced solutions:

```
fitness = 1.0 / (makespan + w_dist * total_distance)    #  
w_dist = 0.01  
  
# measure() returns makespan, total distance, and per-route  
lengths:  
def measure(assignment, order, depot, stops, n_couriers):  
    routes = decode(assignment, order, n_couriers)  
    lengths = np.array([route_length(r, depot, stops) for r in  
                        routes])  
    return float(lengths.max()), float(lengths.sum()), lengths
```

Listing 3 — Fitness and route measurement.

Because fitness is inversely proportional to makespan, maximizing fitness directly minimizes the slowest route. The tiny distance weight (0.01) nudges the search toward globally shorter solutions only when two candidates have the same makespan.

4.3 Selection

Parents are chosen by tournament selection of size k (default 4): k random individuals compete and the fittest wins. Tournament selection gives steady selection pressure without requiring fitness scaling, and the pressure is easily tuned through k .

```
def tournament(population, k, rng):
    picks = rng.choice(len(population), size=k, replace=False)
    return population[max(picks, key=lambda idx:
population[idx].fitness)]
```

Listing 4 — Tournament selection (delivery_ga/engine/operators.py).

4.4 Crossover Operators

Each gene is recombined with an operator suited to its structure:

- Uniform crossover on the assignment gene — a random mask copies each stop's owner from one parent or the other.
- Order crossover (OX, Davis 1985) on the order gene — preserves a contiguous slice from parent A and fills the remaining positions with the missing stops in the order they appear in parent B, producing a valid permutation.

```
def uniform_crossover(assign_a, assign_b, rng):
    mask = rng.random(len(assign_a)) < 0.5
    return np.where(mask, assign_a, assign_b)

def order_crossover(order_a, order_b, rng):
    i, j = sorted(rng.choice(n + 1, size=2, replace=False))
    child = np.full(n, -1)
    child[i:j] = order_a[i:j]                # keep a slice
    from parent A
    used = np.zeros(n, dtype=bool); used[order_a[i:j]] = True
```

```

    fill = order_b[~used[order_b]]          # remaining stops,
parent-B order
    child[:i], child[j:] = fill[:i], fill[i:]
    return child

```

Listing 5 — Uniform and order crossover.

4.5 Mutation Operators

Three mutation operators are applied independently, each with probability `mut_p` (default 0.3):

- Random-reset on assignment — reassign one stop to a different random courier.
- Swap / reverse on order — with equal chance, swap two positions or reverse a short sub-sequence (a 2-opt-like move).
- Relocate (linkage-aware) — moves a stop in both genes at once: it reassigns the stop to a nearby stop's courier and re-inserts it immediately after that neighbor in the order. This single coordinated move crosses fitness valleys that neither single-gene mutation can, directly addressing the epistasis of the two-gene encoding.

```

def mutate_relocate(individual, stops, rng):
    stop_idx = rng.integers(len(individual.assign))
    distances = np.linalg.norm(stops - stops[stop_idx],
axis=1)
    distances[stop_idx] = np.inf
    target = int(rng.choice(np.argsort(distances)[:5])) # a
near neighbour
    individual.assign[stop_idx] = individual.assign[target]
    order = individual.order[individual.order != stop_idx]
    pos = int(np.where(order == target)[0][0])
    individual.order = np.insert(order, pos + 1, stop_idx) #
re-insert beside it

```

Listing 6 — Linkage-aware relocate mutation.

4.6 Evolutionary Loop & Elitism

The algorithm uses generational replacement with elitism. Each generation:

- The top E individuals (default 3) are copied unchanged into the next generation (fitness elitism).
- The remaining slots are filled by breeding: two parents are selected, crossover produces a child, and each mutation operator may fire.
- Optional crossover-credit breeder elitism: when an UN-mutated child beats both its parents, those parents earn credit (decaying $\times 0.5$ each generation). The top breeders by credit survive regardless of their own fitness — selecting on combining ability rather than raw fitness, an analogue of progeny testing in breeding, to counter gene linkage.

```

for _ in range(n_children):
    parent_a = tournament(self.pop, self.tour_k, rng)
    parent_b = tournament(self.pop, self.tour_k, rng)
    child = Individual(
        uniform_crossover(parent_a.assign, parent_b.assign,
rng),
        order_crossover(parent_a.order, parent_b.order,
rng),
    )
    if rng.random() < self.mut_p: mutate_assign(child.assign,
self.k, rng)
    if rng.random() < self.mut_p: mutate_order(child.order,
rng)
    if rng.random() < self.mut_p: mutate_relocate(child,
self.stops, rng)
    self._evaluate(child)

# crossover-credit: reward parents whose unmutated child beat
them both
if child.fitness > max(parent_a.fitness, parent_b.fitness):
    gain = child.fitness - max(parent_a.fitness,
parent_b.fitness)
    parent_a.credit += gain; parent_b.credit += gain

```

Listing 7 — Breeding loop and crossover-credit (delivery_ga/engine/ga.py).

5. Implementation & Architecture

The system is organized as a small Python package, `delivery_ga`, with a clean separation between the problem domain, the GA engine, and the user interface.

Module	Responsibility
<code>domain/maps.py</code>	Map generators (clustered and sanity instances).
<code>domain/routing.py</code>	Chromosome decode, route length, makespan/fitness measurement.
<code>engine/individual.py</code>	Genome container (two genes + cached fitness, makespan, credit).
<code>engine/operators.py</code>	Selection, crossover, and mutation operators.
<code>engine/ga.py</code>	Population initialization, evolutionary loop, elitism, history log.
<code>ui/app.py</code>	Pygame application: setup screen, dashboard, race orchestration.
<code>ui/race.py</code>	Race animation helpers.
<code>ui/widgets.py</code> / <code>theme.py</code>	Buttons, sliders, cards and the visual theme.
<code>ui/sprites.py</code> / <code>assets.py</code>	Asset-backed sprite loaders (truck/house/depot PNGs).
<code>cli.py</code> / <code>main.py</code>	Argument parsing and thin entry point.

The GA engine is independent of the UI and depends only on NumPy, which keeps the optimization core testable and fast. Individuals use `__slots__` and cache their decoded metrics, avoiding repeated allocation across thousands of generations.

6. Visualization (Race Mode)

Every `race_every` generations (default 100), the top distinct candidate solutions are animated as live courier races on the map. Each child solution is drawn as a color family, and each of its couriers as a shade of

that color; a child “finishes” when its slowest truck returns to the depot. Because the slowest truck defines the makespan, the on-screen finishing order matches the fitness ranking — making the abstract objective visually intuitive.

The dashboard also shows, in real time:

- Live makespan of the current best solution.
- Per-courier route-length bars (load balance at a glance).
- A fitness sparkline tracking best/average fitness over generations.
- In-app setup sliders for population, mutation rate, and other parameters.

Interaction keys: SPACE pauses, B toggles the best-routes overlay, S saves a screenshot, and Q quits. World sprites (house, depot, trucks) are pre-rendered PNGs, while all dashboard chrome is drawn at runtime with Pygame primitives.

7. Configuration Parameters

All parameters are exposed via command-line flags and an AppConfig dataclass. Key parameters and their defaults:

Flag	Default	Meaning
--stops / --couriers	100 / 5	Problem size: number of stops and couriers.
--pop / --gens	300 / 1500	Population size and fixed generation count.
--mut	0.3	Per-child, per-operator mutation probability.
--tour-k / --elites	4 / 3	Tournament size and number of fitness elites.
--w-dist	0.01	Total-distance tie-breaker weight in fitness.
--race-every / --top	100 / 5	Race frequency and children shown per

		race.
--credit	on	Toggle crossover-credit breeder elitism.
--sanity	off	Use the obvious-answer validation map.
--seed	0	RNG seed for reproducibility.

Example runs:

```
python main.py                                # default: 100 stops, 5
couriers, race UI
python main.py --sanity                        # obvious-answer
validation map
python main.py --pop 400 --mut 0.3 --seed 7 --race-every 150 -
-credit off
```

8. Validation & Results

Correctness is validated with the sanity map: five tight clusters arranged symmetrically around the depot, one per courier. The obvious optimum is one courier per cluster, and the GA reliably converges toward this assignment, confirming that the encoding, operators, and fitness function cooperate correctly.

On the default clustered instance, the GA progressively reduces the makespan and balances per-courier route lengths. The recorded history (best fitness, average fitness, and best makespan per generation) shows the characteristic GA convergence curve: rapid early improvement followed by gradual refinement. The crossover-credit scheme (--credit on) helps the search escape the epistatic plateaus created by the two coupled genes, complementing the linkage-aware relocate mutation.

Note: quantitative figures depend on the random seed and chosen parameters; the visualization (live makespan, per-courier bars, and the fitness sparkline) is the primary means of observing convergence during a run, and the S key captures screenshots for inclusion in results.

9. Conclusion

This project demonstrates a complete genetic algorithm for a realistic multi-vehicle routing problem, optimizing makespan to balance courier workloads rather than merely minimizing total distance. The two-gene encoding cleanly guarantees feasible solutions, while a set of structure-aware operators — order crossover for the permutation gene and a linkage-aware relocate mutation — address the epistasis inherent in the representation. An optional crossover-credit breeder elitism scheme adds selection on combining ability. Beyond the algorithm, the Pygame race visualization turns an abstract fitness value into an intuitive race, making the system an effective educational demonstration of evolutionary computation. Possible extensions include vehicle-capacity constraints, time windows, asymmetric/road-network distances, and adaptive mutation rates.

10. References

- Dantzig, G. B., & Ramser, J. H. (1959). The Truck Dispatching Problem. *Management Science*, 6(1), 80–91.
- Davis, L. (1985). Applying Adaptive Algorithms to Epistatic Domains. *Proceedings of IJCAI*.
- Holland, J. H. (1975). *Adaptation in Natural and Artificial Systems*. University of Michigan Press.
- Goldberg, D. E. (1989). *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley.